

19. Устойчивост на методите за сортиране. Алгоритми за сортиране чрез сравнения и без сравнения, чрез броене и метод на бройните системи. Сортиране по дялове и чрез разделяне-сливане.

1. Увод

Сортирането е една от най-фундаменталните задачи в компютърните науки - подреждане на елементите на масив в някакъв ред спрямо зададен ключ за сравнение. Значението му произлиза от факта, че сортирани данни позволяват логаритмично търсене (както при двоичното търсене).

Алгоритмите за сортиране се разделят основно по два критерия:

1. **Начин на подреждане** - чрез сравнение на елементи или без сравнение, като се използва допълнителната информация за структурата на ключовете.
2. **Устойчивост** - дали относителният ред на елементи с равни ключове се запазва.

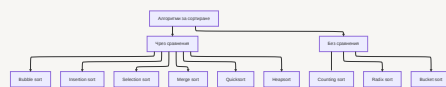
2. Устойчивост

Определение: Сортиращ алгоритъм е устойчив (stable), ако за произволни два елемента a_i и a_j с равни ключове ($key(a_i) = key(a_j)$), при които a_i се намира преди a_j във входната редица, изходната редица също съдържа a_i преди a_j .

Устойчивостта е важна, когато сортираме записи с няколко полета и искаме да запазим предишен ред при сортиране по вторичен ключ. Класически пример: списък от студенти вече сортиран по име; ако след това сортираме по курс, устойчив алгоритъм гарантира, че студентите от един и същи курс остават подредени по име.

Алгоритъм	Устойчив по природа?
Bubble sort	Да
Insertion sort	Да
Selection sort	Не
Merge sort	Да
Quicksort (стандартна реализация)	Не
Heapsort	Не
Counting sort	Да (при коректна реализация)
Radix sort	Да (наследява от стабилна под-процедура)

3. Класификация на алгоритмите за сортиране



4. Долна граница за сортиране чрез сравнение

Всеки алгоритъм от типа "сортиране чрез сравнения" може да се представи като двоично дърво за решения - всеки вътрешен връх отговаря на сравнение между два елемента, а всеки лист отговаря на една от възможните пермутации на входа. Тъй като различните пермутации на n елемента са $n!$, дървото трябва да съдържа поне $n!$ листа.

Двоично дърво с височина h може да има най-много 2^h листа, следователно:

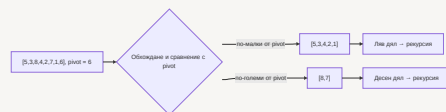
$$2^h \geq n! \implies h \geq \log_2(n!)$$

$$h = \log_2(n!) = \Omega(n \log n)$$

Тъй като височината на дървото за решения отговаря на броя сравнения в най-лошия случай, следва, че **всеки алгоритъм за сортиране чрез сравнения изисква $\Omega(n \log n)$ сравнения** в най-лошия случай. Затова merge sort и heapsort, работещи с $O(n \log n)$, са асимптотически оптимални сред сравнителните алгоритми. Тази граница не важи за алгоритми, които не сравняват елементи директно, а използват допълнителна информация за ключовете.

5. Сортиране по дялове (Quicksort)

Идеята е: Избира се опорен елемент (pivot). Масивът се разделя (partition) на две части: елементи по-малки от pivot и елементи по-големи от него; pivot застава между двете части. Двете части се сортират рекурсивно.



```
int partition(vector<int>& a, int lo, int hi) {
    int pivot = a[hi];
    int i = lo - 1;
    for (int j = lo; j < hi; ++j) {
        if (a[j] < pivot) {
            ++i;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[hi]);
    return i + 1; // финална позиция на pivot
}
```

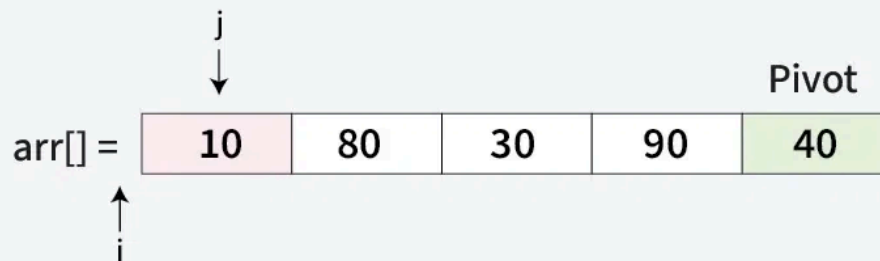
```

void quicksort(vector<int>& a, int lo, int hi) {
    if (lo < hi) {
        int p = partition(a, lo, hi);
        quicksort(a, lo, p - 1);
        quicksort(a, p + 1, hi);
    }
}

```

01
Step

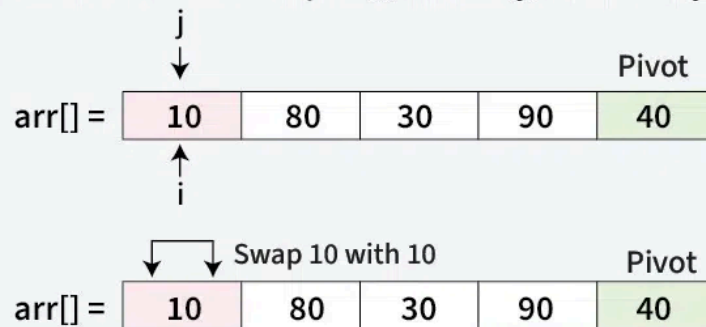
Pivot Selection: The last element $\text{arr}[4] = 40$ is chosen as the pivot.
Initial Pointers: $i = -1$ and $j = 0$.



Lomuto Partition Algorithm

02
Step

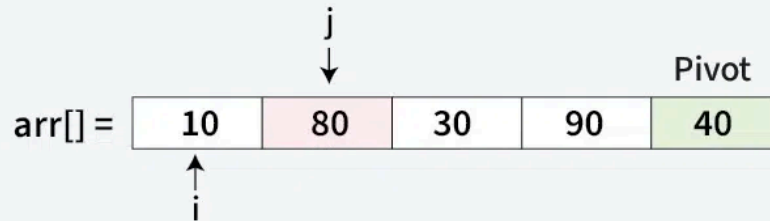
Since, $\text{arr}[j] < \text{pivot}$ ($10 < 40$)
Increment i to 0 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Lomuto Partition Algorithm

03
Step

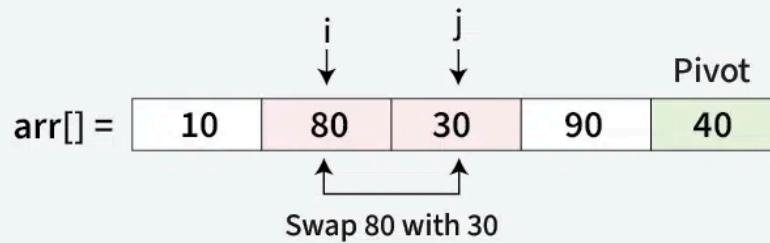
Since, $\text{arr}[j] > \text{pivot}$ ($80 < 40$)
No swap needed. Increment j by 1



Lomuto Partition Algorithm

04
Step

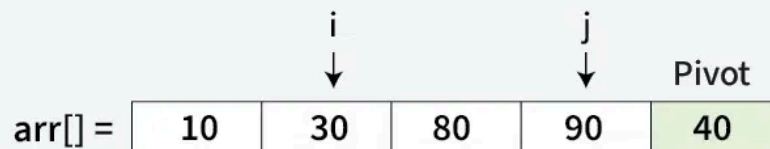
Since, $\text{arr}[j] < \text{pivot}$ ($30 < 40$)
Increment i by 1 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Lomuto Partition Algorithm

05
Step

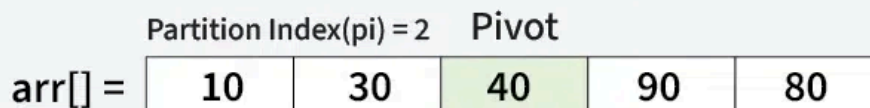
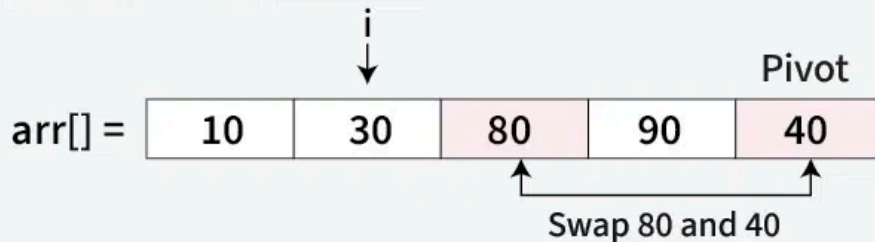
Since, $\text{arr}[j] > \text{pivot}$ ($90 < 40$)
No swap needed. Increment j by 1



Lomuto Partition Algorithm

06
Step

Since traversal of j has ended. Now move pivot to its correct position, Swap $arr[i + 1] = arr[2]$ with $arr[4] = 40$.



Lomuto Partition Algorithm

Анализ на сложността. Ако partition разделя масива балансирано (например наполовина), рекурентата е $T(n) = \Theta(n \log n)$.

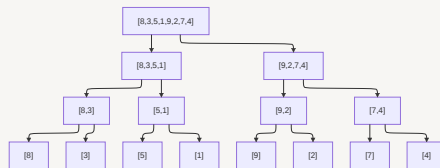
При най-неблагоприятен избор на pivot (например винаги най-малкия или най-големия елемент - случва се при вече сортиран вход и "тъп" избор на pivot), рекурентата дегенерира и е $\Theta(n^2)$.

Средният случай е $\Theta(n \log n)$. За да се избегне влошаване при вече подреден или враждебно подготвен вход, на практика се използва случаен pivot или медиана от три елемента.

Свойства. Quicksort е **сортиране на място** (in-place), с логаритмична допълнителна памет за стека на рекурсията, но **не е устойчив** - размените (swap) по време на partition могат да променят относителния ред на елементи с равни ключове.

6. Сортиране чрез разделяне и сливане (Merge sort)

Идеята е: Класически алгоритъм по парадигмата "разделяй и владей": масивът се разделя рекурсивно на две половини, докато остане по един елемент (тривиално сортиран), след което половините се сливат (merge) в сортиран ред.



След разделянето следва обратен ход, при който съседните листа се сливат нагоре по дървото: $[8] + [3] \rightarrow [3, 8]$, $[5] + [1] \rightarrow [1, 5]$, после $[3, 8] + [1, 5] \rightarrow [1, 3, 5, 8]$, аналогично за дясната половина, и накрая двете сортирани половини се сливат в целия резултат.

```
void merge(vector<int>& a, int lo, int mid, int hi) {
    vector<int> tmp;
    int i = lo, j = mid + 1;
```

```

while (i <= mid && j <= hi) {
    if (a[i] <= a[j]) tmp.push_back(a[i++]); // "<=" пази устойчивостта
    else tmp.push_back(a[j++]);
}
while (i <= mid) tmp.push_back(a[i++]);
while (j <= hi) tmp.push_back(a[j++]);
for (int k = lo; k <= hi; ++k) a[k] = tmp[k - lo];
}

void mergesort(vector<int>& a, int lo, int hi) {
    if (lo < hi) {
        int mid = (lo + hi) / 2;
        mergesort(a, lo, mid);
        mergesort(a, mid + 1, hi);
        merge(a, lo, mid, hi);
    }
}

```

Анализ на сложността. Всяко ниво на рекурсията извършва общо $\Theta(n)$ работа за сливане, а дълбочината е $\log_2 n$.

$$T(n) = \Theta(n \log n)$$

За разлика от quicksort, тази сложност важи във всички случаи - най-добър, среден и най-лош, защото разделянето винаги е балансирано наполовина, независимо от входните данни.

Свойства. Merge sort е устойчив, ако при равенство на ключове $a[i] \leq a[j]$ първо се взима елементът от лявата половина. Не е сортиране на място в класическата реализация - изисква допълнителна памет $O(n)$ за временния масив.

7. Сортиране без сравнение

Долната граница $\Omega(n \log n)$ важи само за модела "сортиране чрез сравнения". Ако се знае допълнителна структурна информация за ключовете (например че са цели числа в ограничен диапазон), може да се постигне линейно време $\Theta(n)$, тъй като алгоритъмът не сравнява елементи директно, а ги разпределя по позиция.

7.1 Сортиране чрез броене (Counting sort)

Идеята е: Приложимо, когато ключовете са цели числа в диапазон $[0, k]$. Преброява се колко пъти се среща всяка стойност, изчислява се кумулативна сума, за да се определи крайната позиция на всеки елемент, и накрая елементите се преместват в изходния масив, обхождайки входа **отзад напред** - това е ключово за запазването на устойчивостта.

```

void countingSort(vector<int>& a, int k) {
    vector<int> count(k + 1, 0);
    for (int x : a) count[x]++;
    for (int i = 1; i <= k; ++i) count[i] += count[i - 1];
}

```

```

vector<int> output(a.size());
for (int i = (int)a.size() - 1; i >= 0; --i) { // отзад напред → устойчивост
    output[--count[a[i]]] = a[i];
}
a = output;
}

```

Сложност: $\Theta(n + k)$. Ако $k = O(n)$, това е $\Theta(n)$ – линейно време.

01
Step

Find out the maximum element from the given array.

	0	1	2	3	4	5	6	7	Max
arr[]=	2	5	3	0	2	3	0	3	5

03
Step

In the cntArr[], store the count of each unique element of the input array at their respective indices.

	0	1	2	3	4	5
cntArr[]=	2	0	2	3	0	1

04
Step

Store the cumulative sum or prefix sum of the elements of the cntArr[] by doing $\text{cntArr}[i] = \text{cntArr}[i - 1] + \text{cntArr}[i]$.

	0	1	2	3	4	5
cntArr[]=	2	2	4	7	7	8

05
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[i]] - 1] = \text{arr}[i]$ and decrement $\text{cntArr}[\text{arr}[i]]$. Traverse the input array in reverse to maintain the order of equal elements, ensuring the sort remains stable.

	0	1	2	3	4	5	6	7
arr[]=	2	5	3	0	2	3	0	3

↓ 3

	0	1	2	3	4	5	6	7
cntArr[]=	2	2	4	7	7	8		

↓ 7-1=6

	0	1	2	3	4	5	6	7
ans[]=							3	

12
Step

Update $\text{ans}[\text{cntArr}[\text{arr}[0]] - 1] = \text{arr}[0]$
Also, update $\text{cntArr}[\text{arr}[0]] = \text{cntArr}[\text{arr}[0]] - 1$

	0	1	2	3	4	5	6	7
arr[]=	2	5	3	0	2	3	0	3

↓ 2

	0	1	2	3	4	5	6	7
cntArr[]=	0	2	3	4	7	7		

↓ 2

	0	1	2	3	4	5	6	7
ans[]=	0	0	2	2	3	3	3	5

7.2 Метод на бройните системи (Radix sort)

Идеята е: Прилага се, когато ключовете могат да се представят като числа с d цифри в дадена бройна система с основа k (например десетична, $k = 10$).

Сортира се последователно по всяка цифра, като се тръгва от най-младшата цифра (LSD - least significant digit) към най-старшата, използвайки стабилен алгоритъм (обикновено counting sort) за всяка цифрова позиция.

Ключово свойство: тъй като под-процедурата за сортиране по единична цифра е устойчива, а обработката минава от младша към старша цифра, крайният резултат е коректно сортиран по цялото число - елементите, равни по старшите цифри, остават в реда, определен от предходните (по-младши) проходи.

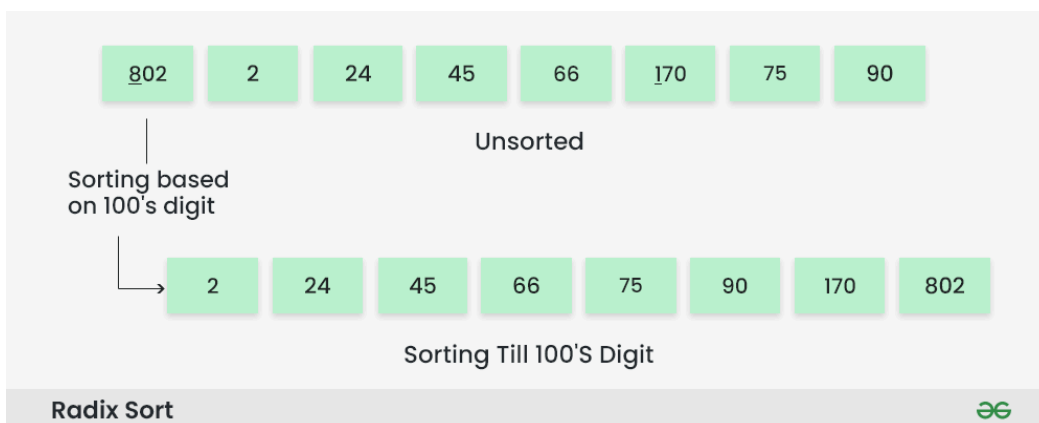
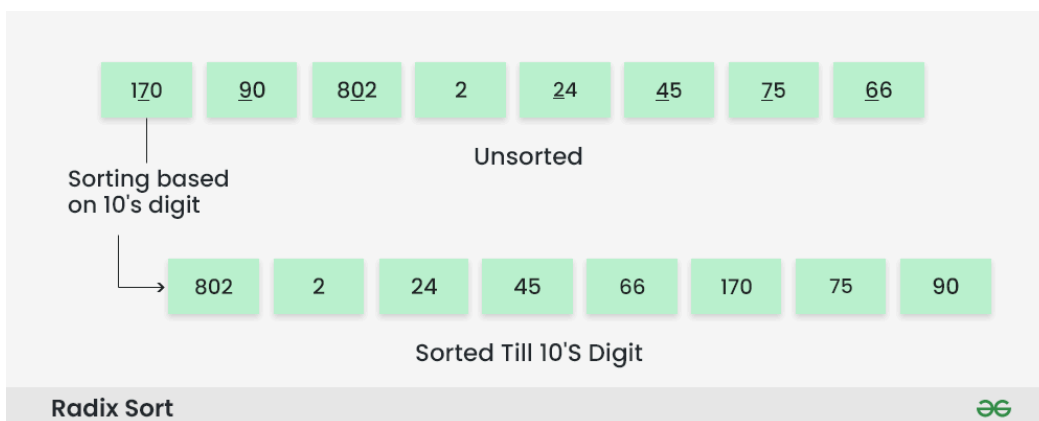
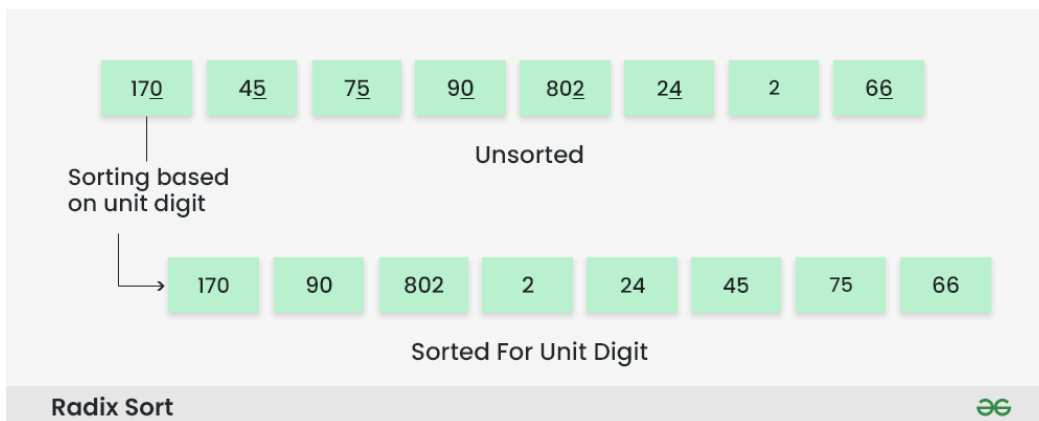


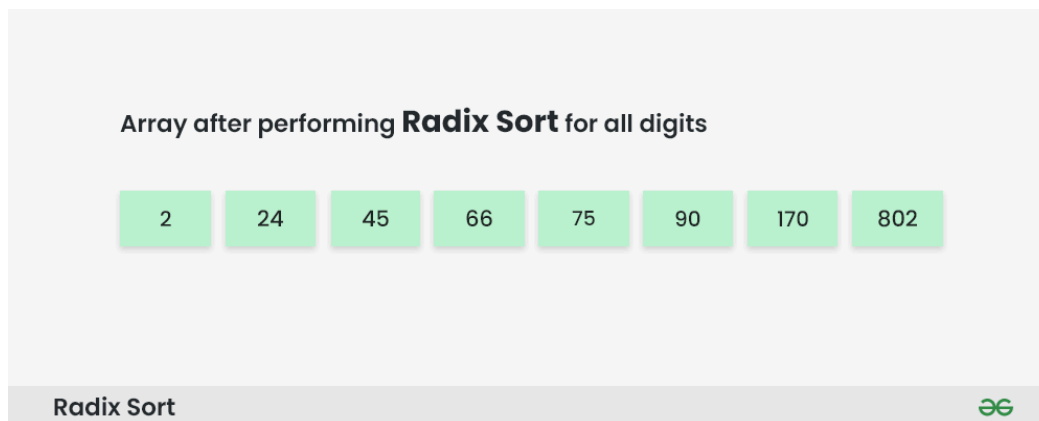
```
int getDigit(int num, int place) {
    return (num / place) % 10;
}

void countingSortByDigit(vector<int>& a, int place) {
    vector<int> output(a.size());
    vector<int> count(10, 0);
    for (int x : a) count[getDigit(x, place)]++;
    for (int i = 1; i < 10; ++i) count[i] += count[i - 1];
    for (int i = (int)a.size() - 1; i >= 0; --i) {
        int d = getDigit(a[i], place);
        output[--count[d]] = a[i];
    }
    a = output;
}

void radixSort(vector<int>& a) {
    int maxVal = *max_element(a.begin(), a.end());
    for (int place = 1; maxVal / place > 0; place *= 10)
        countingSortByDigit(a, place);
}
```

Сложност: $\Theta(d(n + k))$, където d е броят цифри, а k е основата на бройната система. Ако $d = O(1)$ и $k = O(1)$, общата сложност е $\Theta(n)$.





8. Обобщителна таблица

Алгоритъм	Най-добър	Среден	Най-лош	Допълнителна памет	Устойчив	На място
Bubble sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да	Да
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Да	Не
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ стек	Не	Да
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$O(n+k)$	Да	Не
Radix sort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$O(n+k)$	Да	Не